

Temas Tratados en el Trabajo Práctico 3

- Estrategias de búsqueda local.
- Algoritmos Evolutivos.
- Problemas de Satisfacción de Restricciones.

Ejercicios Teóricos

1. ¿Qué mecanismo de detención presenta el algoritmo de Ascensión de Colinas?
Describe el problema que puede presentar este mecanismo y cómo se llaman las áreas donde ocurren estos problemas.

El algoritmo de Ascensión de Colinas (Hill Climbing) es una técnica de búsqueda local que consiste en moverse siempre hacia el vecino que representa una mejora en la función objetivo o heurística. El criterio de detención es justamente cuando ya no existen vecinos que mejoren el estado actual: en ese momento, el algoritmo se detiene porque interpreta que llegó a un "óptimo".

El problema es que este mecanismo de detención puede resultar engañoso. Muchas veces el algoritmo no se encuentra realmente en la mejor solución posible (óptimo global), sino en un punto que es mejor que sus vecinos inmediatos pero que está lejos de la mejor solución total. A esto se le conoce como quedar atrapado en un máximo local.

Además de los máximos locales, aparecen otras dificultades: por ejemplo, las mesetas o planicies, que son regiones del espacio de estados donde muchos vecinos tienen exactamente el mismo valor, y entonces el algoritmo no tiene criterio claro para decidir hacia dónde moverse. Otro caso problemático son las crestas, que son zonas donde la mejoría se da solo si se avanza en dos direcciones combinadas, pero el algoritmo examina de a una, por lo que puede quedar atascado.

2. Describa las distintas heurísticas que se emplean en un problema de Satisfacción de Restricciones.

En los CSP lo que se busca es asignar valores a variables de modo que se cumplan ciertas restricciones (por ejemplo, colorear un mapa o un rompecabezas sin que dos piezas adyacentes tengan el mismo color). Dado que los espacios de búsqueda suelen ser muy grandes, se emplean heurísticas que ayudan a decidir qué variable elegir y qué valor probar primero, con el fin de reducir al máximo la cantidad de retrocesos (backtracking). Una de las más utilizadas es la heurística MRV (Minimum Remaining Values), también conocida como "fail-first". La idea es elegir primero la variable que tenga menos opciones de valores disponibles en su dominio. Esto se hace porque, si una variable va a

provocar un fallo, es mejor detectarlo cuanto antes y no perder tiempo explorando ramas inútiles.

Cuando varias variables empatan en MRV, se puede usar la heurística de grado como criterio de desempate. En este caso se selecciona la variable que esté más "conectada" en el grafo de restricciones, es decir, la que participe en más relaciones con otras variables no asignadas. Así se prioriza resolver primero aquellas decisiones que impactan en más partes del problema.

Por otro lado, una vez elegida la variable, también es útil decidir en qué orden probar los valores. Ahí entra la heurística del valor menos restrictivo (Least Constraining Value, LCV). Lo que se busca es asignar primero el valor que elimine menos opciones de los vecinos, manteniendo abiertas más posibilidades para el resto de las variables.

Finalmente, existen mecanismos complementarios como el Forward Checking, que consiste en ir reduciendo los dominios de las variables vecinas a medida que se hacen asignaciones, descartando valores que ya no pueden ser válidos. Yendo un paso más allá, se puede aplicar consistencia de arcos (como en el algoritmo AC-3), donde se fuerza a que cada valor de una variable tenga al menos un valor compatible en los dominios de sus vecinos. En conjunto, todas estas heurísticas buscan lo mismo: guiar la búsqueda hacia asignaciones más prometedoras y detectar los callejones sin salida lo antes posible, logrando una resolución mucho más eficiente que el simple backtracking ciego.

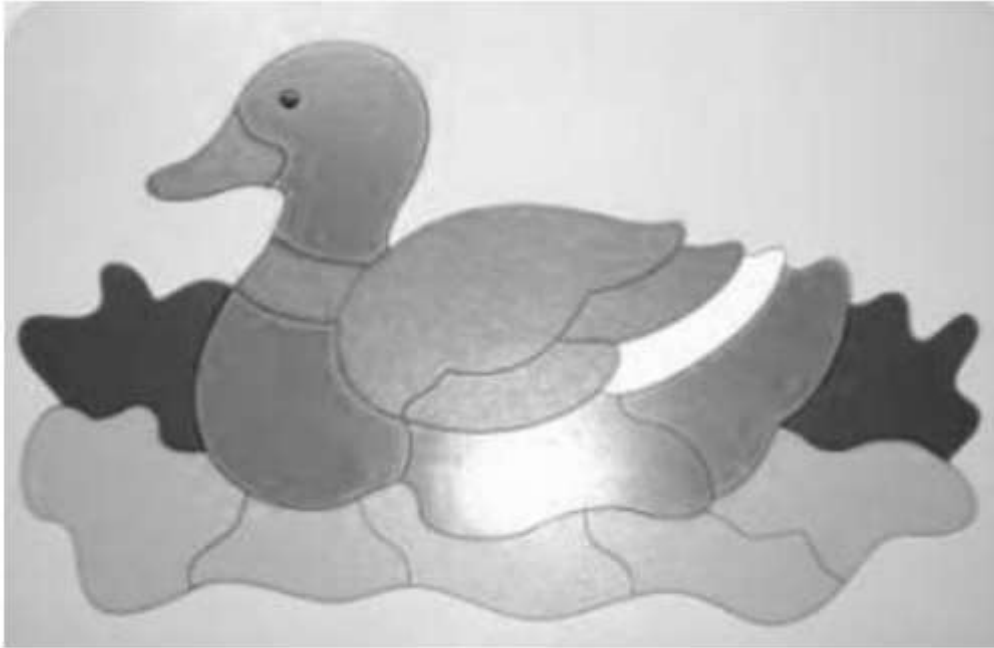
3. Se desea colorear el rompecabezas mostrado en la imagen con 7 colores distintos de manera que ninguna pieza tenga el mismo color que sus vecinas. Realice en una tabla el proceso de una búsqueda con Comprobación hacia Adelante empleando una heurística del Valor más Restringido.

```
In [2]: import requests
        from PIL import Image
        from io import BytesIO
        import matplotlib.pyplot as plt

        # URL directa de Google Drive
        url = "https://drive.google.com/uc?export=view&id=1j94jFVxVG9y_ZnrMW0scQGb2MZ0Cd"

        # Descargar la imagen
        response = requests.get(url)
        img = Image.open(BytesIO(response.content))

        # Mostrar la imagen
        plt.imshow(img)
        plt.axis('off') # Ocultar ejes
        plt.show()
```



Para este ejercicio numeramos las partes del rompecabezas y le asignamos un valor numérico empezando por aquellas partes que restringen a más vecinos como 1. A partir de ahí lo que tenemos que hacer es siempre elegir como paso siguiente aquella parte que esté más restringida por nuestras elecciones previas, como en el caso de 13 que cuando fue pintada tenía 2 restricciones frente a las demás partes que solo tenían como máximo 1.

En el .pdf adjuntado en el repositorio se encuentra la tabla con los procedimientos para la asignación de colores de la imagen.

Ejercicios de Implementación

4. Encuentre el máximo de la función $f(x) = \frac{\sin(x)}{x+0.1}$ en $x \in [-10; -6]$ con un error menor a 0.1 utilizando el algoritmo *hill climbing*.

El presente trabajo desarrolla una aplicación en Python que implementa el algoritmo de Hill Climbing para la búsqueda de máximos locales en una función matemática.

Utilizamos la función:

$$f(x) = \sin x / (x + 0.1)$$

La aplicación cuenta con una interfaz gráfica construida con Tkinter y Matplotlib, permitiendo al usuario definir un punto inicial y visualizar el resultado del algoritmo.

2. Descripción del Algoritmo Hill Climbing

El Hill Climbing es un algoritmo de optimización iterativo que comienza en un punto inicial y se desplaza hacia el vecino que produce la mayor mejora de la función objetivo.

Pasos principales:

1. Seleccionar un punto inicial aleatorio o definido por el usuario. (Al comienzo es aleatorio)
2. Evaluar la función en el punto actual y en sus vecinos ($x + \text{paso}$ y $x - \text{paso}$).
3. Moverse al vecino con mayor valor de función.
4. Repetir hasta que:

No haya mejoras, el error sea menor a un umbral, o se alcance un número máximo de iteraciones. Particularmente, en esta implementación se usa un paso de 0.01 y un criterio de parada de error < 0.0001 o 10,000 iteraciones.

3. Desarrollo de la Aplicación

3.1 Función Objetivo

```
def f(x): return math.sin(x) / (x + 0.1)
```

3.2 Algoritmo Hill Climbing

```
def HILL_CLIMBING(funcion, comienzo, paso=0.01, error=0.0001): ... return x, y
```

El algoritmo evalúa vecinos y selecciona el de mayor valor hasta converger.

3.3 Interfaz Gráfica

Se construyó una clase HillClimbGUI que hereda de tk.Tk e integra:

Panel de control con entrada de punto inicial.

Botón para ejecutar el cálculo.

Gráfico de la función con Matplotlib embebido en Tkinter.

Visualización del punto máximo encontrado en el gráfico con una X roja.

3.4 Visualización

La función se dibuja en el intervalo $[-10, 10]$.

El máximo local encontrado se marca en el gráfico.

Los resultados también se muestran en forma de texto en la interfaz.

4. Resultados

Ejecutando el programa desde distintos puntos iniciales, se observa que:

El algoritmo converge a máximos locales, que dependen del punto de inicio.

La interfaz permite repetir el cálculo fácilmente probando diferentes condiciones.

La representación gráfica facilita comprender la dinámica de Hill Climbing.

Pero cabe aclarar que por la naturaleza de la función objetivo, si nuestro comienzo está entre un valor cercano a -4 y antes de 0, el máximo se queda muy cerca de 0 pero llega a tener valores de función enormes porque la función tiende a infinito por izquierda de 0.

5. Conclusiones

La implementación demuestra el funcionamiento del algoritmo Hill Climbing en la búsqueda de máximos locales.

La interfaz gráfica en Tkinter ofrece una herramienta didáctica y visual para experimentar con el algoritmo.

Se observa la principal limitación del método: la convergencia depende fuertemente del punto inicial, lo cual puede llevar a diferentes máximos locales.

Una mejora futura sería implementar Random Restarts Hill Climbing, que realiza múltiples ejecuciones desde puntos iniciales distintos para incrementar la probabilidad de encontrar el máximo global.

5. Diseñe e implemente un algoritmo de Recocido Simulado para que juegue contra usted al Ta-te-ti. Varíe los valores de temperatura inicial entre partidas, ¿qué diferencia observa cuando la temperatura es más alta con respecto a cuando la temperatura es más baja?

Algoritmo de recocido simulado

Resumen alto nivel

Este es un juego de Ta-Te-Ti con interfaz en Pygame donde el jugador es 'X' y la computadora 'O'. La IA elige su jugada usando una versión experimental de Recocido Simulado (Simulated Annealing) combinada con heurísticas críticas (victoria/bloqueo) cuando la "temperatura" es baja. El programa tiene:

- configuración y constantes;
- lógica del juego (ganador/empate);
- heurística de evaluación del tablero;
- función de recocido simulado para elegir movimiento;
- componentes de interfaz (botones, selector dificultad);
- funciones de dibujo y animación;
- bucle principal que maneja eventos y coordina todo.

Imports y configuración global

- `import pygame, sys, random, math` etc.: dependencias estándar.

- Constantes visuales y de juego: WIDTH, BOARD_SIZE, PANEL_H, colores, CELL = BOARD_SIZE // 3, FPS, ANIM_PLACEMENT_MS (200 ms para animación de aparición), AI_DELAY_MS (500 ms retardo antes de que la IA juegue).
- Se inicializa Pygame (pygame.init()), se crea screen, clock y las fuentes (font_large, font_mid, ...).

Estas constantes controlan tamaño, timing y aspecto. Cambiarlas afectará la UI/velocidad.

Enum de dificultades

```
class Difficulty(Enum): EASY = ("Fácil", 4.5, EASY_COLOR) MEDIUM = ("Medio", 2.0, MEDIUM_COLOR) HARD = ("Difícil", 0.3, HARD_COLOR)
```

- Cada nivel contiene una etiqueta, una temperatura inicial temperature (T0) y un color.
- Interpretación: T0 alto = más exploración (IA más aleatoria / más débil); T0 bajo = menos exploración (IA más "determinista" / fuerte). Esa temperatura se pasa luego al recocido.

Lógica básica del juego

Funciones:

- WIN_LINES: todas las combinaciones ganadoras (filas, columnas, diagonales).
- check_winner(board) -> Optional[("X"|"O", trio)]: revisa cada trio; devuelve jugador ganador y el trio si hay una línea completa; devuelve None si no.
- is_draw(board) -> bool: True si no quedan celdas vacías.

Estas funciones son triviales y correctas: trabajan en la representación board: List[str] con 9 elementos " (vacío), 'X' o 'O'.

Heurística: evaluate_board_detailed(board) -> int

Objetivo: dar un puntaje numérico al tablero desde la perspectiva de 'O' (positivo favorece a O; negativo favorece a X).

Pasos importantes:

1. Revisa el ganador inmediato: +1000 si 'O' ganó, -1000 si 'X' ganó. Esa es una recompensa terminal grande que domina decisiones.
2. Para cada línea (trio) calcula:
 - o_count, x_count, empty_count
 - Si o_count == 2 and empty_count == 1: score += 60 (oportunidad de victoria)
 - Si x_count == 2 and empty_count == 1: score -= 50 (amenaza crítica del rival)

- Si `o_count == 1` and `empty_count == 2`: `score += 8` (desarrollo)
- Si `x_count == 1` and `empty_count == 2`: `score -= 5`

3. Bonus por posiciones:

- Centro (`board[4]`): +15 si O, -10 si X (centro muy valioso).
- Esquinas (0,2,6,8): +7 por cada esquina de O, -5 por cada esquina de X.

Racional: los pesos (60,50,8,5,15,7, etc.) priorizan ganar/bloquear y prefieren centro/esquinas. Estos números son empíricos: ajustándolos cambias el comportamiento de la IA.

Recocido simulado: `simulated_annealing_move_experimental(...)`

Esta es la parte clave. Voy a descomponerla paso a paso.

Firma y parámetros

```
def simulated_annealing_move_experimental(board, T0, alpha=0.90, T_min=0.01,
max_iter=1000)
```

- `board`: estado actual (sin la jugada de la IA todavía).
- `T0`: temperatura base tomada del selector de dificultad.
- `alpha`: factor de enfriamiento por iteración (geometrico). Aquí 0.90 enfría relativamente rápido.
- `T_min`: umbral para detener.
- `max_iter`: límite de iteraciones internas del bucle de recocido.

Flujo interno (explicado)

1. `empties = [i for i ... if v == ""]`: lista de celdas libres. Si no hay, retorna `None`.
2. Heurísticas críticas a baja temperatura: si `T0 < 1.0`, el código intenta:
 - Victoria inmediata: para cada hueco prueba `test[i] = 'O'` y si produce ganador devuelve ese `i`. (Juega la victoria inmediata).
 - Bloqueo crítico: para cada hueco prueba `test[i] = 'X'`; si X ganaría al jugar ahí, bloquea con prob. 90% (`random < 0.9`). Esto asegura que a bajas temperaturas la IA prioriza taponar amenazas y ganar.
3. Recocido puro:
 - `current_move = random.choice(empties)` — se elige una posición inicial aleatoria.

- `current_score = evaluate_board_detailed(board[:])` — observación importante: aquí se evalúa el tablero sin colocar O en `current_move`. (Ver nota de mejora abajo.)
- $T = T_0 * 20.0$ — escalado de la temperatura para darle más rango. Esto multiplica la T_0 elegida por 20.

Bucle `for _ in range(max_iter):`

- Si $T < T_{\min}$ rompe.
- `neighbor_move = random.choice(emptyies)` — candidato vecino.
- `neighbor_board = board.copy(); neighbor_board[neighbor_move] = 'O'`
- `neighbor_score = evaluate_board_detailed(neighbor_board)`
- `delta = neighbor_score - current_score`
- Criterio de aceptación:
- Si $\delta \geq 0$: acepta (mejora o igual).
- Si $\delta < 0$: acepta con prob. $\exp(\min(500, \delta / \max(0.01, T)))$.
- δ/T es negativo; $\exp(\text{neg}) < 1$, menor cuanto mayor la diferencia negativa.
- `min(500, ...)` sirve para proteger la exponenciación de valores extremos.
- Si se acepta: `current_move = neighbor_move; current_score = neighbor_score`
- Enfriamiento: $T *= \alpha$ (multiplicativo)
- Al terminar devuelve `current_move`.

Interpretación matemática

- El algoritmo busca moves que incrementen la heurística. Las peores jugadas se aceptan raramente, dependiendo de T . Si T es grande (fácil), se aceptan más malas jugadas (IA más aleatoria). Si T es pequeña (difícil), la prob. de aceptar una jugada peor es baja y el algoritmo tiende a elegir movimientos con mejor heurística.
- Notar que `current_score` inicialmente es la evaluación del tablero sin O añadido; por tanto δ para el primer vecino será $\text{score}(\text{board with neighbor O}) - \text{score}(\text{board})$, lo cual es coherente si tratas `current_score` como referencia base. Sin embargo, si se pretende hacer recocido entre soluciones (tableros con una O), lo más correcto sería definir `current_board = board` con `current_board[current_move] = 'O'` y calcular su score. Esto es una observación de diseño (ver sugerencias abajo).

7. Pequeñas precisiones sobre la fórmula de aceptación

- Aceptación `random.random() < math.exp(min(500, delta / max(0.01, T)))`:
- $\max(0.01, T)$ evita dividir por cero o por T muy pequeña.

- `min(500, ...)` evita pasar a `math.exp` un argumento demasiado grande (overflow).
- Para `delta >= 0` ya aceptas automáticamente; la exponencial solo se usa cuando `delta < 0`.
- En resumen: es la clásica $\exp(\Delta E/T)$, aplicada de forma robusta.

8. Componentes UI y eventos

`DifficultySelector` (dataclass) Parámetros de posición y tamaño, `selected` es el `Difficulty` actual, `enabled` indica si se puede cambiar (se desactiva tras iniciar la partida).

`get_button_rect(difficulty)` calcula rectángulo de botón según índice.

`handle_event(event)`: si clic dentro de un botón y `enabled`, cambia `selected`.

`draw(surf)`: dibuja etiqueta "Dificultad:", luego los tres botones con colores y estilos según estado (`selected`, `enabled`). También muestra `T0` junto a botones.

`Button` (dataclass) Un botón genérico con `rect` y `text`.

`draw` dibuja `rect` y texto.

`clicked(pos)` chequea colisiones.

9. Dibujo y animaciones Funciones para renderizado: `cell_center(idx)`: devuelve el centro (x,y) de la celda `idx`.

`draw_grid(highlight_hover=True)`: pinta fondo, dibuja una celda resaltada si el cursor está encima y las cuatro líneas gruesas de la grilla.

`draw_piece_X(center, scale)` y `draw_piece_O(center, scale)`: dibujan X u O de tamaño proporcional a `scale` (0..1). Esto permite animación de aparición (trazo creciente / radio creciente).

`draw_board_pieces(board, appear_time, now_ms)`: para cada celda con ficha calcula $\text{progress} = (\text{now} - \text{t0}) / \text{ANIM_PLACEMENT_MS}$ y llama a `draw_piece_X/O` con ese `scale`.

`draw_win_line(trio, now_ms, start_ms)`: hace animación de línea ganadora desde centro de la primera celda a la tercera, interpolando según progreso.

`is_board_empty(board)`: función utilitaria (no fundamental en flujo principal).

El mecanismo de animación es: al colocar una ficha se guarda `appear_time[idx] = now` (timestamp); mientras $\text{now} - \text{t0} < \text{ANIM_PLACEMENT_MS}$ la pieza se dibuja parcialmente.

10. Bucle principal (`main()`) — estado y flujo

Variables de estado principales:

- `board` (lista de 9 strings), `appear_time` (lista de timestamps), `game_over`, `winner_info`, `win_anim_start`.

- `game_started` controla si la partida se inició (caso en que se bloquea selector de dificultad).
- `player_turn` indica si toca al usuario, `ai_scheduled_at` guarda el timestamp en el que la IA debe jugar (delay visual).

Flujo del bucle:

1. Manejo de eventos Pygame:

QUIT sale.

`difficulty_selector.handle_event(event)` permite cambiar dificultad antes de empezar.

MouseDown:

Si clic en `btn_reset`: reinicia todo (vuelve a permitir selector).

Si clic dentro del tablero y `player_turn` y celda vacía:

Si primera jugada, deshabilita selector.

Coloca 'X' y `appear_time[idx] = now`.

Comprueba ganador o empate (usando `check_winner/is_draw`).

Si sigue: `player_turn = False` y `ai_scheduled_at = now + AI_DELAY_MS` (programa la IA).

2. Ejecución IA: cuando `now >= ai_scheduled_at`:

`T0 = difficulty_selector.selected.temperature`

`move = simulated_annealing_move_experimental(board, T0=T0)`

Si `move` no `None`: coloca 'O', `appear_time[move] = now`

Chequea ganador/empate; si no terminó, `player_turn = True`

`ai_scheduled_at = None`

3. Dibujo: `draw_grid`, `draw_board_pieces`, si `game_over` muestra `draw_win_line`, pinta el panel inferior con estado ("Tu turno (X)", "Pensando...", "¡Ganaste!", etc.), dibuja selector y botón reiniciar.

4. `pygame.display.flip()` y `clock.tick(FPS)`.

5. Conclusiones

En este algoritmo de recocido simulado se puede observar cómo el comportamiento del algoritmo difiere cuando variamos la temperatura inicial. El comportamiento de la IA es más 'torpe' a medida que la temperatura sube, ya que la probabilidad de aceptar una casilla cualquiera es más elevada.

6. Diseñe e implemente un algoritmo genético para cargar una grúa con $n = 10$ *cajas* que puede soportar un peso máximo $C = 1000$ *kg*. Cada caja j tiene asociado un precio p_j y un peso w_j como se indica en la tabla de abajo, de manera que el algoritmo debe ser capaz de maximizar el precio sin superar el límite de carga.

Elemento (j)	1	2	3	4	5	6	7	8	9	10
Precio (p_j)	100	50	115	25	200	30	40	100	100	100
Peso (w_j)	300	200	450	145	664	90	150	355	401	395

6.1 En primer lugar, es necesario representar qué cajas estarán cargadas en la grúa y cuáles no. Esta representación corresponde a un Individuo con el que trabajará el algoritmo.

6.2 A continuación, genere una Población que contenga un número N de individuos (se recomienda elegir un número par). Es necesario crear un control que verifique que ninguno de los individuos supere el peso límite.

6.3 Cree ahora una función que permita evaluar la Idoneidad de cada individuo y seleccione $N/2$ parejas usando el método de la ruleta.

6.4 Por último, Cruce las parejas elegidas, aplique un mecanismo de Mutación y verifique que los individuos de la nueva población no superen el límite de peso.

6.5 Realice este proceso iterativamente hasta que se cumpla el mecanismo de detención de su elección y muestre el mejor individuo obtenido junto con el peso y el precio que alcanza.

1. Problema y Objetivo

El programa implementa un Algoritmo Genético (AG) para resolver el problema de la mochila 0/1. En nuestro caso, la "mochila" es una grúa. El objetivo es seleccionar un subconjunto de ítems (10 en total) de manera que se **maximice el precio total** sin superar la capacidad máxima de la grúa **(1000 kg)**.

2. Representación de Soluciones

Cada solución (individuo) se representa como un vector binario de longitud 10: 1 → el ítem está incluido en la mochila. 0 → el ítem no está incluido. Ejemplo: [1, 0, 1, 0, 0, 1, 0, 0, 1, 0] → se incluyen los ítems 1, 3, 6 y 9.

3. Inicialización de la Población

El programa genera $N = 50$ individuos iniciales de forma aleatoria. Para asegurar validez, cada individuo es reparado usando el criterio de ratio precio/peso, eliminando ítems de

baja eficiencia hasta que no supere la capacidad de la mochila.

4. Función de Fitness

El fitness mide la calidad de cada individuo: Si el peso total ≤ 1000 kg $\rightarrow$ fitness = precio total. Si excede el peso $\rightarrow$ se aplica una penalización proporcional al exceso de peso. Esto asegura que las soluciones válidas sean siempre favorecidas.

5. Operadores Genéticos

Selección: método de ruleta (individuos con mayor fitness tienen más probabilidad de ser elegidos, aunque todos pueden participar). Cruce: one-point crossover. Se elige un punto aleatorio y se intercambian segmentos entre dos padres. Mutación: bit-flip mutation. Con probabilidad $p_m = 0.1$, cada gen puede invertirse ($0 \leftrightarrow 1$). Reparación: se eliminan ítems de menor ratio precio/peso si el hijo es inválido. Elitismo: se preserva el mejor individuo de cada generación ($k = 1$).

6. Ciclo Evolutivo

El algoritmo repite el siguiente ciclo durante $G = 100$ generaciones: Evaluar el fitness de la población. Seleccionar padres por ruleta. Aplicar cruce y mutación. Reparar soluciones inválidas. Aplicar elitismo. Registrar estadísticas de mejor y promedio fitness.

7. Verificación por Fuerza Bruta

Como el problema tiene solo $2^{10} = 1024$ combinaciones posibles, el programa implementa también una búsqueda exhaustiva (fuerza bruta) para encontrar el óptimo exacto. Esto permite comparar el desempeño del AG con la solución óptima real.

8. Análisis de Resultados y Gráfica

Se encuentran en el pdf del jupyter

9. Conclusiones El programa implementa todos los componentes fundamentales de un algoritmo genético: representación binaria, fitness con penalización, selección por ruleta, cruce, mutación, reparación y elitismo. La comparación con fuerza bruta valida la calidad de las soluciones obtenidas. La gráfica de evolución muestra claramente el efecto del elitismo (curva azul estable) y la exploración de la población (curva roja variable). En este caso, el algoritmo logró alcanzar la solución óptima, demostrando su efectividad para este problema.

Pruebas adicionales. Nos dio la sensación de que el algoritmo funciona demasiado bien. Esto se debe a varios factores, como el elitismo y la reparación por ratio, la cual se aplica en la generación de cada individuo (asegurando de que partimos de un individuo válido $C < 1000$ kg y descartando los "peores" integrantes de los individuos) y en cada hijo generado (luego de la mutación). Es decir, esta reparación por ratio fuerza muy bruscamente la "solución". Esto puede generar estancamientos en los máximos locales. Para tratar de evitar esto, se incluyen mutación y cruce. De todas formas, se incluye realizaron algunas pruebas cambiando tres parámetros principales: elitismo, reparación por ratio y tamaño de la población.

Bibliografía

Russell, S. & Norvig, P. (2004) *Inteligencia Artificial: Un Enfoque Moderno*. Pearson Educación S.A. (2a Ed.) Madrid, España

Poole, D. & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press (3a Ed.) Vancouver, Canada